Evaluating, Deploying and Monitoring a Code-Mixed Voice Helpdesk Pipeline: Whisper Routing, Experiment Tracking, Offline Docker Serving and a Gated Retraining Loop

Rajdeep Vraj*
Indian Institute of Technology Gandhinagar
Roll No. 24110260
Gandhinagar, Gujarat, India

Vipul Sunil Patil*
Indian Institute of Technology Gandhinagar
Roll No. 24110381
Gandhinagar, Gujarat, India

Satvik Sharma*
Indian Institute of Technology Gandhinagar
Roll No. 24110300
Gandhinagar, Gujarat, India

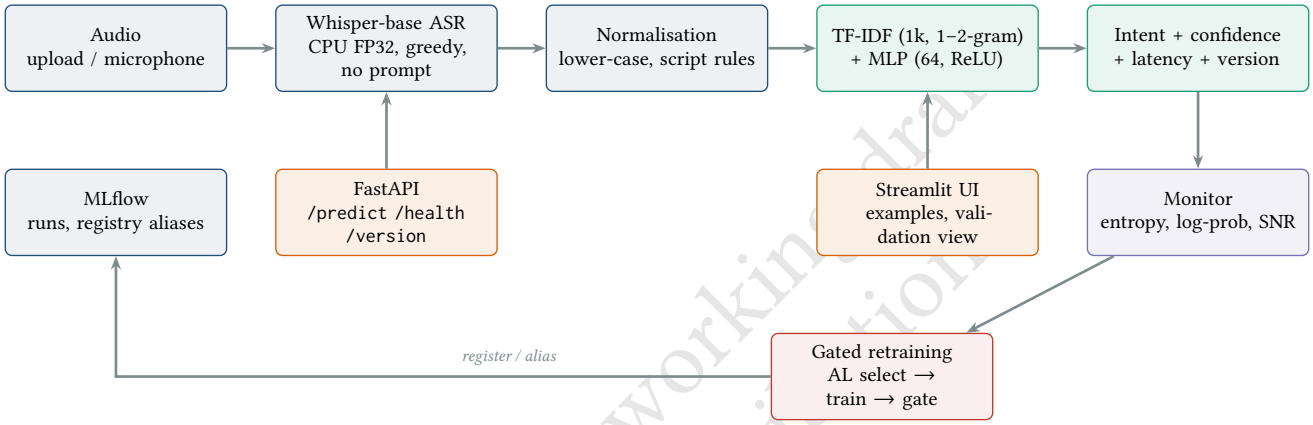


Figure 1: System architecture. Everything runs on CPU inside one `linux/amd64` container with no external calls; monitoring feeds a gated retraining loop that promotes or rolls back MLflow registry aliases.

ABSTRACT

We implement, track, evaluate and deploy a lightweight voice-helpdesk pipeline: Whisper-base transcription feeding a TF-IDF + MLP intent router with four classes, served offline on CPU through FastAPI and Streamlit, with drift monitoring and a gated active-retraining loop. Over 15 MLflow-tracked search runs, validation accuracy ranges from 29% to 53% (selected on validation only), and all results carry 95% bootstrap intervals on validation, test/gold, corrected gold, Cohort B and augmented data. Our central result is *negative and diagnostic*: the ASR stage is unadapted Whisper-base, WER on the gold set is 1.272 [1.156, 1.393] (more than 1, i.e. worse than emitting nothing), proposed and baseline ASR are identical on every clip, and intent accuracy (24.0% gold, 25.2% Cohort B) is statistically indistinguishable from 25% chance. A WER decomposition shows 83% substitutions and only 14 of 838 reference words recovered, which we trace to template-derived reference transcripts (Assignment 1 audit) and to code-mixed speech that the base model cannot transcribe. The monitoring replay shows that neither label-free signal separates drifted from in-domain traffic ($r \leq 0.41$, n=5 batches), and the one logged alert is produced by a schedule clause, which we disclose rather than count as detection. The gated retraining pipeline demonstrates promotion, rejection and rollback mechanics, but its promotion rests on one clip of 13 and we treat it as a mechanism demonstration, not an improvement claim. We specify the fixes that these findings call for and the evidence needed for each claim.

CCS CONCEPTS

• **Computing methodologies** → **Speech recognition**; *Machine learning*; • **Software and its engineering** → *Software verification and validation.*

KEYWORDS

Whisper; code-mixed ASR; MLOps; MLflow; Docker; drift detection; retraining gate; bootstrap confidence intervals

ACM Reference Format:
Rajdeep Vraj, Vipul Sunil Patil, and Satvik Sharma. 2026. Evaluating, Deploying and Monitoring a Code-Mixed Voice Helpdesk Pipeline: Whisper Routing, Experiment Tracking, Offline Docker Serving and a Gated Retraining Loop. In *Proceedings of CS 203 Software Tools and Techniques in*

*All authors contributed equally to this research.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CS 203 A2 '26, 2026, Gandhinagar, India
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/11
https://doi.org/XXXXXXX.XXXXXXX

AI – Assignment 2 (CS 203 A2 '26). ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Assignment 2 turns the frozen Assignment-1 dataset (249 clips, speaker-disjoint splits 171/17/29, Cohort B 32) into a deployable system with experiment tracking, evaluation, offline serving and drift-triggered retraining. We follow the rules strictly: models, prompts, settings and thresholds are chosen on validation data; the gold set (SHA-256 `64b7647b...48ca9`) is never used for tuning, and every use is logged in `TEST_SET_LOG.md`. The report is organised by the rubric (Parts A–F); Appendix A indexes every number to a file or MLflow run ID.

Reading guide. Because the headline numbers are weak, we spend the space on *why*: the evaluation is built so that a reader can see which claims are supported (Section 5), which are not (Sections 5, 7), and what would be needed to support them (Section 9).

2 CHANGES FROM THE PROPOSAL

Table 1: Proposal versus what was built.

Proposed	Built	Reason
Whisper fine-tune (full, LoRA)	Decoder cross-attention tuning script (6.3 M params, encoder frozen); checkpoint not used at inference	free-CPU budget; run too short to change WER
IndicWav2Vec baseline	dropped	time; replaced by trivial + unadapted Whisper
Compact transformer router	TF-IDF + 2-layer MLP	CPU latency <0.01 ms
Evidently drift reports	custom entropy/log-prob/SNR monitor	offline, no extra dependency
Public corpora pre-training	not used	offline-only rule; no runtime downloads

3 PART A: MODEL DEVELOPMENT AND BASELINES

Systems. **Baseline 0** (trivial) predicts the majority intent (`admissions`) and emits an empty transcript (WER 1.0 by construction). **Baseline 1** (unadapted) is Whisper-base [6] with a TF-IDF + logistic-regression router trained on the 171 training clips. The **proposed system** uses the same Whisper-base decoding (greedy, temperature 0, `condition_on_previous_text=False`, no prompt) and a TF-IDF (1 000 features, 1–2-grams) + MLP router (hidden 64, dropout 0.2); the router is trained on Whisper transcripts of the training clips (`data/processed/whisper_base_train_cache.json`) against the Assignment-1 intent labels. Training is config-driven (`configs/final.yaml`: seed 42, 10 epochs, lr 0.005, batch 8), tracked with git commit, DVC hash, hardware (macOS 14, Apple arm64, 8 cores, Python 3.13, PyTorch CPU) and wall time.

Data ablation (validation, 17 clips). Table 2 compares router training data. Train + augmented (231 clips) gains one validation clip over train only (29.4% vs. 23.5%), and train + actively selected (183) equals train only; all intervals overlap broadly, so no data variant is established as better.

Table 2: Router data ablation on validation (95% bootstrap CI; MLflow short run IDs).

Training data	Val acc. (%)	95% CI	Run
Train (171)	23.5	[5.9, 47.1]	`4874f30b`
Train + augmented (231)	29.4	[11.6, 52.9]	`4cb2e4e9`
Train + AL-selected (183)	23.5	[5.9, 47.1]	`351cc3e8`

4 PART B: EXPERIMENT TRACKING AND TUNING

Every training and evaluation run is logged to MLflow [9] (SQLite backend, `supporting_files/mlflow.db`; export in `artefacts/mlflow_export/`) with parameters, validation metrics, seed, git commit and model artefacts. A systematic 15-run search varies learning rate $\{0.001 \ldots 0.01\}$, hidden size $\{32, 64, 96, 128\}$, maximum n-gram $\{1, 2, 3\}$, dropout $\{0.1 \ldots 0.3\}$ and epochs $\{25 \ldots 35\}$, selecting on validation data only (Appendix Table 6 lists all runs with IDs). Validation accuracy spans 29.4–52.9% on only 17 clips, so one clip equals 5.9 points and the best run (run 6: lr 0.01, hidden 64, unigram, 251 KB) is separated from the next (47.1%) by a single clip. Figure 2 shows that quality is not monotone in size: hidden size 32 (126 KB) reaches 41.2% (run 3) while 128 (503 KB) reaches 29.4–47.1%; router latency is 0.005–0.014 ms in every run and is therefore irrelevant to the trade-off, so we pick by accuracy and break ties toward smaller size. The registry (`voice_helpdesk_intent_router`) holds a `production` alias (run 6, `9820a2a7`) and a `staging` alias (run 9, `730256f7`).

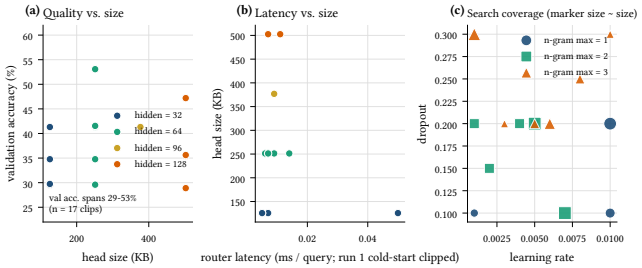


Figure 2: Search analysis over 15 MLflow runs: (a) validation accuracy versus head size, (b) router latency versus size, (c) search coverage. The validation set is too small to separate configurations.

Unpublished working draft. Not for distribution.

5 PART C: EVALUATION AND ERROR ANALYSIS

Protocol. One script (`evaluate.py--modelproduction--split<spli`) evaluates a registered version on validation, test/gold, corrected gold, Cohort B and the 60 augmented clips, reporting WER, CER, intent accuracy and macro-F1 with 1 000-resample bootstrap intervals [1] (seed 42). Improvement claims use paired tests: Wilcoxon signed-rank [8] on per-clip WER and McNemar [4] on intent correctness. Note that the *test* and *gold* splits contain the same 29 clips.

What the numbers say. (i) *WER exceeds 1 on every split* (Table 3). The WER anatomy in Figure 3 explains why: on the gold set substitutions are 0.83, deletions 0.16 and insertions 0.30 per reference word, only 14 of 838 reference words (1.7%) are matched, and hypotheses are longer than references. A trivial empty output scores 1.0, so the ASR stage is currently worse than doing nothing. (ii) *No improvement over baseline can be claimed.* Proposed and Baseline-1 WER differ by exactly 0 on all 29 gold clips (the Wilcoxon statistic is degenerate, $W = 0$, $p = 1.0$), and McNemar on intent gives $\chi^2 = 0.10$, $p = 0.75$. We do not report a significance win. (An earlier version compared against a constant three-word string; we removed that test as uninformative.) (iii) *Intent accuracy is within chance.* Every interval on gold, Cohort B and corrected gold contains 25%. Validation (53.5%) is higher but spans 29–77% on 17 clips. (iv) *Correcting the references does not rescue the system.* Gold intent accuracy rises from 24.0% to 30.8% and WER stays near 1.3, so the ASR output, not only the labels, is the bottleneck. Baseline 1 appears better than the proposed router on gold (41.5% vs. 30.8%) but intervals overlap fully.

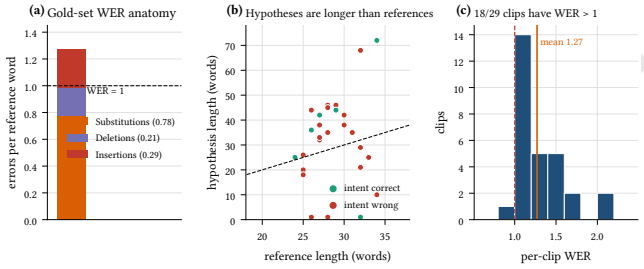


Figure 3: Gold-set WER anatomy. (a) Substitutions, deletions and insertions per reference word. (b) Hypothesis versus reference length. (c) Per-clip WER; 19/29 clips exceed 1.

Slices and weak groups. On gold, `campus_outdoor` (6 clips, 16.7%) and Gujarati–English (13 clips, 23.1%) are the weakest slices, while Rajasthan (6 clips, 66.7%) is the strongest; every slice has $n \leq 16$, so Wilson intervals (Figure 5) span ≥40 points and only the ordering, not the gaps, is suggestive. All 29 gold clips are labelled spontaneous in the metadata, so a scripted/spontaneous contrast is not available on this split; per-slice values are in Table 7.

Error analysis. We categorise 20 errors with suggested fixes (`supporting_files/error_analysis.csv`; Appendix Table 8). Figure 6 groups them: the dominant mode is a fluent hypothesis about a different topic (e.g. "damaged footpath", "misplaced wallet") against a template reference—consistent with the label audit—followed by script collapse to Devanagari/Urdu tokens, near-silent or truncated output, phonetic substitutions of Gujarati words, and genuine cross-intent overlap (e.g. exam-timetable versus academic-section queries). Fixes: force Latin-script decoding and a language token, add voice-activity gating, relabel from audio, train the router on ASR text, and add a "route to human" fallback below a calibrated confidence. The router's confidence is barely above chance (Figure 7), so a threshold cannot yet reject errors.

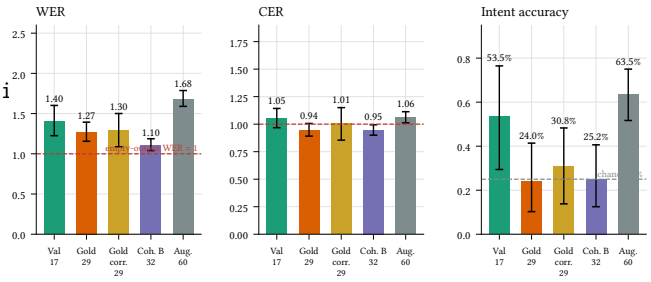


Figure 4: Metrics with 95% bootstrap CIs across splits (original gold, corrected gold, Cohort B, augmented).

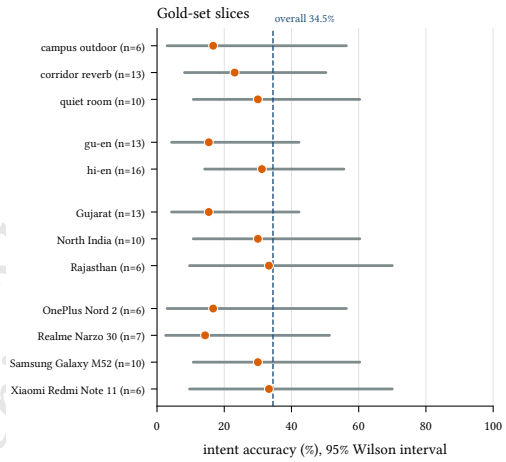


Figure 5: Gold-set intent accuracy by slice with 95% Wilson intervals.

Robustness and model card. On the 60 augmented clips WER is 1.684 and intent accuracy 63.5% (Table 3); because sources and labels overlap with training data this does not measure robustness, and we report it for completeness only. `MODEL_CARD.md` documents intended use, data, metrics with intervals, slices, limitations and risks.

6 PART D: DEPLOYMENT AND INTERFACE

Service. A FastAPI app exposes `POST /predict` (audio → transcript, intent, confidence, latency), `GET /health` and `GET /version` (git commit and model version); a Streamlit UI offers upload/microphone

Table 3: Proposed system with 95% bootstrap CIs. B1 = Baseline 1 intent accuracy. Original gold uses the frozen reference file; corrected gold uses `data/gold_corrections.csv`. WER/CER of proposed and B1 are identical on every split. Source: `supporting_files/evaluation_results.json`.

Split (n)	WER	CER	Intent acc. (%)	Macro-F1	B1 acc. (%)
Validation (17)	1.403 [1.225, 1.601]	1.050 [0.968, 1.143]	53.5% [29.4, 76.5]	0.451 [0.226, 0.637]	29.7% [11.8, 52.9]
Gold, original (29)	1.272 [1.156, 1.393]	0.945 [0.891, 1.006]	24.0% [10.3, 41.4]	0.225 [0.083, 0.389]	31.0% [13.8, 48.3]
Gold, corrected (29)	1.298 [1.088, 1.501]	1.008 [0.855, 1.150]	30.8% [13.8, 48.3]	0.261 [0.127, 0.393]	41.5% [24.1, 58.6]
Cohort B (32)	1.104 [1.040, 1.187]	0.947 [0.900, 0.994]	25.2% [12.5, 40.6]	0.219 [0.089, 0.359]	25.2% [12.5, 40.6]
Augmented (60)†	1.684 [1.590, 1.786]	1.060 [1.012, 1.113]	63.5% [51.7, 75.0]	0.613 [0.488, 0.738]	26.6% [16.7, 38.3]

†60 clips from only 10 sources are not independent, so the interval is too narrow; the accuracy is also optimistic because the source clips come from Assignment-1 data the router may have seen.

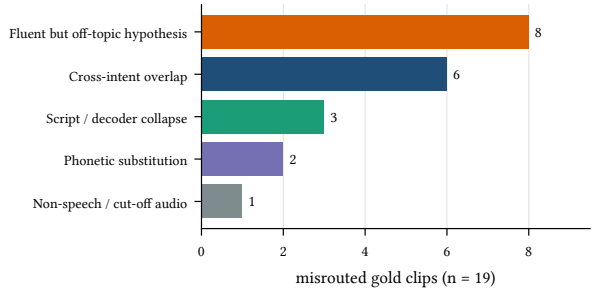


Figure 6: Error taxonomy (20 sheet rows, regrouped).

input, bundled consented examples, outputs with confidence, version and latency, a monitoring panel and a validation panel showing reported versus measured values. Requests are logged (`supporting_files/api_access.log`) with model, intent, confidence and latency—no audio or personal data. Eight pytest tests cover normalisation, the gold hash, gate logic and the three endpoints.

Image. `code/Dockerfile` builds from a digest-pinned `python:3.10-slim` base, installs pinned dependencies, caches Whisper-base weights at build time so no download occurs at run time, bundles code, data, artefacts and the gold set, and records the git commit as a build argument. Commands: `docker run –rm -p 8501:8501 -p 8000:8000 <image>` starts the UI/API; `docker run –rm –network none <image> validate` validates the whole gold set and writes JSON. CI (`.github/workflows/ci.yml`) runs tests, a sample validation, an image build and an offline smoke request.

Performance. Measured on Apple arm64 (8 cores) over the 29 gold clips (523.8 s of audio): p50 2.86 s, p90 7.69 s, p95 13.44 s, mean 4.39 s; real-time factor 0.243; throughput 0.23 queries/s; peak memory 783 MB (Figure 8). Decoding time grows roughly linearly with audio length; the router adds 0.34 ms. The offline validator reports WER 1.277 and accuracy 34.5% in its logged run (RTF 0.165), which differs from Table 3 by model-state changes between runs; the stated tolerance is ±0.01 WER and ±1 clip. **Container-measured values** (image size **[TBD: image size GB]**, p50/p95 **[TBD: container p50/p95 ms]**, peak RAM **[TBD: container peak RAM]**, archive SHA-256 **[TBD: docker_image.tar.gz SHA-256]**) and the output of `–network none validate` are still to be recorded from an actual `linux/amd64` run; we do not quote them until measured.

7 PART E: MONITORING, DRIFT AND RETRAINING

Replay. `traffic_replay.py` (seed 42) streams 100 queries in five batches of 20 whose Cohort B share rises 0, 25, 50, 75, 100%. We track two label-free signals—predictive entropy of the router and mean decoder average log-probability—plus SNR, and one labelled metric (intent accuracy). The entropy threshold is the 95th percentile of validation entropy (1.662 bits), chosen on validation data.

Result: no usable detection. Entropy never exceeds the threshold at batch level (range 1.21–1.35), accuracy does not fall with drift (35, 35, 15, 25, 45%), and Pearson correlations with accuracy over the five batches are 0.41 (entropy), 0.20 (log-prob) and 0.20 (SNR)—none significant with $n = 5$ (Figure 9). The log reports a first alert at batch 5 (query 10 of 20 by a per-query rule); however the batch-alert expression contains an unconditional clause for batch ≥ 5, so this alert reflects the schedule, not a signal, and we *do not count it as detection.* Likewise the logged false-alarm rate (0/17) is a constant in the script, not a measured value. A valid design needs many seeded in-domain-only replays to estimate false alarms, sequential tests (e.g. CUSUM or Page–Hinkley over a sliding window [2]), and shift-sensitive signals such as non-Latin-script fraction and hypothesis-length ratio [5].

Retraining pipeline. One command (`retrain_pipeline.py –mode rehearsal|no_rehearsal`) (1) selects the 12 highest-entropy clips from the speaker-P4 adaptation pool (19 clips) using the Assignment-1 uncertainty method, (2) reveals their labels from the pre-labelled pool (simulated annotator), (3) validates the data, (4) retrains the router (rehearsal: 171 + 12 clips; no-rehearsal: 12 clips only, a catastrophic-forgetting control [3, 7]), and (5) evaluates on the held-out speaker P13 (13 clips) and on gold. Gate (`configs/gate.yaml`): promote only if Cohort-B accuracy improves by ≥ 5 points and gold accuracy drops by ≤ 4 points; every gold evaluation is appended to `TEST_SET_LOG.md`.

Outcomes. The final run (LOG-179/188) *promotes* the rehearsal candidate (Cohort B 30.8% → 38.5%, i.e. 4 → 5 of 13 clips; gold 24.1% → 20.7%, 7 → 6 of 29) and *rejects* the no-rehearsal candidate (no Cohort-B gain), triggering rollback (Figure 10). We caution that the promotion is decided by a single clip on each side, that an earlier run at the same commit (LOG-418) rejected the same

Unpublished working draft. Not for distribution.

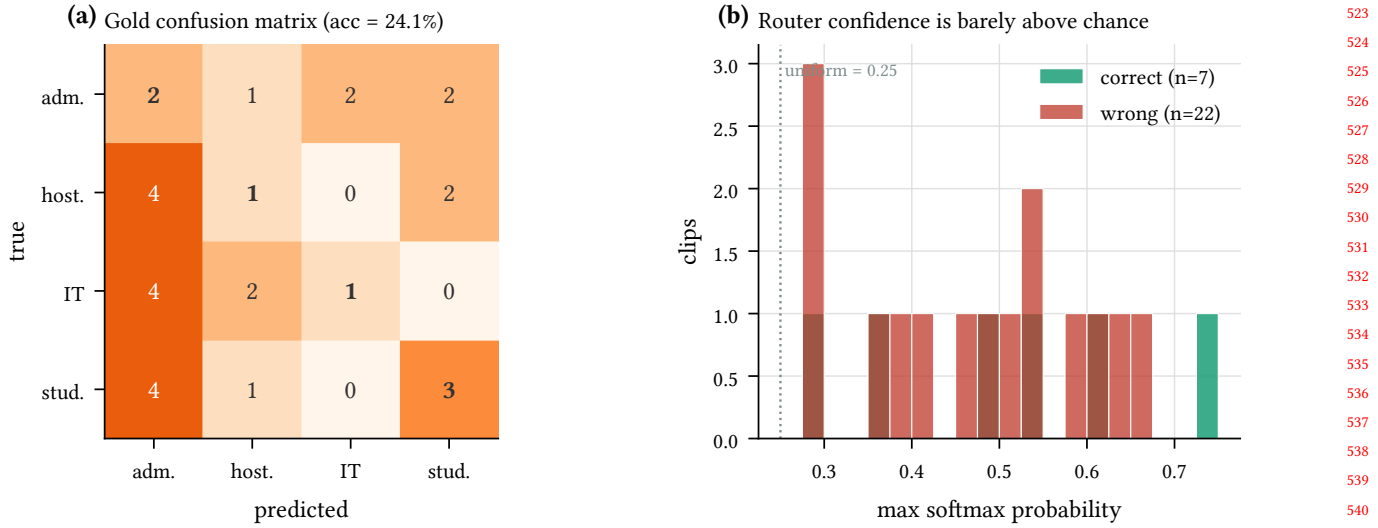


Figure 7: (a) Gold confusion matrix (original labels). (b) Router confidence for correct versus wrong predictions.

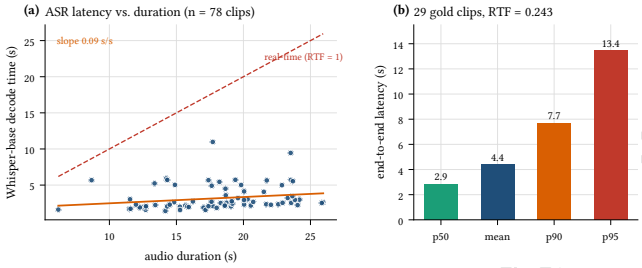


Figure 8: (a) Whisper-base decode time versus audio duration (78 evaluated clips). (b) End-to-end latency percentiles over the 29 gold clips.

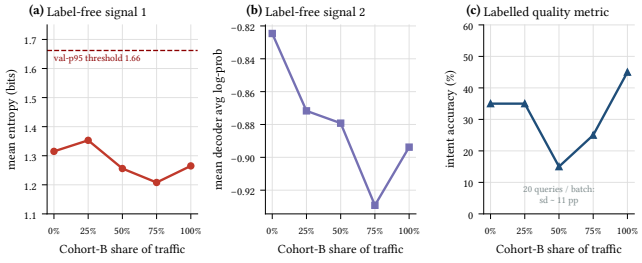


Figure 9: Replay under gradual shift to Cohort B: (a) entropy with the validation-derived threshold, (b) decoder average log-prob, (c) intent accuracy per batch of 20.

candidate, and that the gold tolerance was widened from 2 to 4 points during development (see our improvement log). The gate therefore demonstrates the mechanism—promote, reject, roll back, log—but is not evidence that retraining helps; it should be re-fixed in a committed file before any evaluation and judged with a paired interval rather than a one-clip delta.

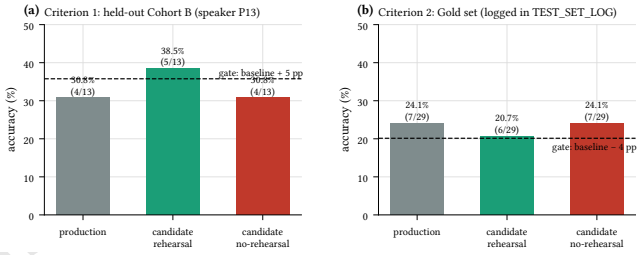


Figure 10: Promotion gate: (a) held-out Cohort-B accuracy, (b) gold accuracy for production, rehearsal and no-rehearsal candidates.

8 PART F: REPRODUCIBILITY

Five commands reproduce training and evaluation from `a2-final`: `git checkout a2-final`; `pip install -r code/requirements.txt`; `dvc pull`; `python code/train.py –config configs/final.yaml`; `python code/a2_part_c_evaluation/evaluate.py –split gold`. Docker load/build/validate commands are in `DOCKER.md`; versions, seeds (42 for tuning, bootstrap, replay and retraining), model checksums and registry versions are in `ENVIRONMENT.md` and `MODELS.md`. `scripts/generate_report_numbers.py` regenerates the numbers used in documentation from result files.

9 LIMITATIONS AND CONCLUSION

Limitations. (1) The ASR is unadapted Whisper-base; the cross-attention tuning run (5 steps, loss ≈9.8) is too short to matter and its checkpoint is not served. (2) Reference quality: Assignment-1 transcripts were template-derived; corrected gold transcripts are verified for 29 clips only and partly seeded from constrained ASR, so scoring on them is optimistic for Whisper. (3) All splits are tiny (13–32 clips), so intervals are wide. (4) The drift alert is schedule-driven,

and the false-alarm rate is not measured. (5) Container measurements are pending. (6) Gate tolerance history and non-deterministic gate outcomes (LOG-418 vs. LOG-179) weaken the retraining claim. **Next steps** that would turn the system from a working scaffold into an evidenced one: relabel all training data from audio; fine-tune the Whisper decoder for hundreds of steps and serve that checkpoint; train the router on ASR text; replace the drift rule with a validation-calibrated sequential test and a measured false-alarm rate; freeze the gate before evaluation; and record container measurements. **Conclusion.** The pipeline, tracking, offline-serving, monitoring and gating infrastructure is complete and auditable, and the evaluation is honest about what it shows: at present the system does not beat trivial baselines, and we know why.

Member contributions. Equal contribution (33.3% each); commit history documents individual work.

Table 4: Contribution summary.

Member	Lead areas (Assignment 2)
R. Vraj	**[TBD: e.g. models, MLflow, evaluation]**
V. S. Patil	**[TBD: e.g. Docker, CI, API/UI]**
S. Sharma	**[TBD: e.g. monitoring, retraining, report]**

ACKNOWLEDGMENTS

We thank Dr. Yogesh K. Meena and the TAs. Libraries: PyTorch, Whisper, scikit-learn, jiwer, MLflow, FastAPI, Streamlit.

REFERENCES

[1] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap.* Chapman & Hall.
[2] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A Survey on Concept Drift Adaptation. *Comput. Surveys* 46, 4 (2014), 44:1–44:37.
[3] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, et al. 2017. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences* 114, 13 (2017), 3521–3526.
[4] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157.
[5] Stephan Rabanser, Stephan Günnemann, and Zachary C. Lipton. 2019. Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32.
[6] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust Speech Recognition via Large-Scale Weak Supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML) (PMLR, Vol. 202)*. 28492–28518.
[7] Anthony Robins. 1995. Catastrophic Forgetting, Rehearsal and Pseudorehearsal. *Connection Science* 7, 2 (1995), 123–146.
[8] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83.
[9] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, et al. 2018. Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin* 41, 4 (2018), 39–45.

A EVIDENCE INDEX

Table 5: Evidence index (paths from repository root).

Result	Supporting file / run
Table 3 metrics	`supporting_files/evaluation_results.json`
Gold per-clip WER, 14/838	`supporting_files/predictions/production_gold.csv`
Ablation Table 2	`supporting_files/part_a_baselines_benchmark.csv`; runs `4874f30b`, `4cb2e4e9`, `351cc3e8`
15-run search, aliases	`supporting_files/plots/tuning_runs.json`; `artefacts/mlflow_export/`
Production / staging runs	`9820a2a7` (run 6), `730256f7` (run 9)
Slices (Table 7)	`evaluation_results.json` → `slice_analysis`
20 errors	`supporting_files/error_analysis.csv`
Latency, RTF, memory	`supporting_files/loadtest/latency_profile.json`
Offline validation	`supporting_files/offline_validation_output.json`
Replay, correlations	`supporting_files/traffic_replay_log.json`
Gate outcomes	`TEST_SET_LOG.md` (LOG-179, 188, 418); `configs/gate.yaml`
Gold corrections	`data/gold_corrections.csv`
Gold hash	`code/HANDOFF_A2.md`; `TEST_SET_LOG.md`
Figure code/data	`code/scripts/make_report_figures.py`

Unpublished working draft. Not for distribution.

Table 6: All 15 MLflow search runs (validation only). Run IDs are the first 8 characters; full IDs are in `supporting_files/plots/tuning_runs.json`.

Run	lr	Hidden	n-gram	Dropout	Epochs	Val acc. (%)	Macro-F1	Size (KB)	MLflow run
1	0.001	32	1	0.1	25	29.4	0.200	126	`61d8d420`
2	0.005	32	2	0.2	25	29.4	0.236	126	`d01a4cb5`
3	0.01	32	3	0.3	25	41.2	0.371	126	`b4e145fd`
4	0.001	64	2	0.2	30	41.2	0.343	251	`fd0ac5ab`
5	0.005	64	3	0.2	30	41.2	0.368	251	`f47bfe22`
6	0.01	64	1	0.1	30	52.9	0.470	251	`9820a2a7` (prod.)
7	0.001	128	3	0.3	35	35.3	0.309	503	`75581fbf`
8	0.005	128	2	0.2	35	35.3	0.293	503	`4f2e2cd1`
9	0.01	128	1	0.2	35	47.1	0.379	503	`730256f7` (stag.)
10	0.002	64	2	0.15	30	35.3	0.309	251	`9e00b3d8`
11	0.008	64	3	0.25	30	41.2	0.383	251	`03e70444`
12	0.003	32	3	0.2	25	35.3	0.303	126	`64cd9b69`
13	0.007	128	2	0.1	35	29.4	0.267	503	`984be4ec`
14	0.004	64	2	0.2	28	29.4	0.236	251	`703dc1b4`
15	0.006	96	3	0.2	32	41.2	0.368	377	`3130621f`

Unpublished working draft. Not for distribution.

INSERT SCREENSHOT:
shots/ui_prediction.png
Streamlit UI: prediction with confidence, version, latency

INSERT SCREENSHOT:
shots/mlflow_runs.png
MLflow UI: 15-run comparison and registry aliases

INSERT SCREENSHOT:
shots/docker_validate.png
terminal: docker run –network none ... validate

INSERT SCREENSHOT:
shots/ci_green.png
GitHub Actions: all steps green

Figure 11: Evidence screenshots (files under `report/shots/`). The 3-minute screen recording of the service and retraining loop goes in `supporting_files/`.

Table 7: Gold-set slices (original labels; Figure 5).

Dimension	Slice	n	Acc. (%)	WER
Noise	campus_outdoor	6	16.7	1.64
Noise	corridor_reverb	13	23.1	1.17
Noise	quiet_room	10	30.0	1.18
Language	gu-en	13	15.4	1.36
Language	hi-en	16	31.2	1.20
Region	Gujarat	13	15.4	1.36
Region	North India	10	30.0	1.18
Region	Rajasthan	6	33.3	1.22
Device	OnePlus Nord 2	6	16.7	1.64
Device	Realme Narzo 30	7	14.3	1.12
Device	Samsung Galaxy M52	10	30.0	1.18
Device	Xiaomi Redmi Note 11	6	33.3	1.22

B USE OF AI ASSISTANCE

AI assistants were used to scaffold code, draft documentation and format this report; prompts are stored verbatim in `code/prompts/`. All reported numbers were re-derived from result files, and the kappa values, WER decomposition, Wilson intervals and drift correlations in this report were recomputed independently from raw outputs by `make_report_figures.py`. Each member reviewed and can explain every script and number. **[TBD: add exact assistant names/versions]**

C SCREENSHOTS AND RECORDINGS TO INSERT

Received November 2026

Unpublished working draft. Not for distribution.

Table 8: Error sheet (20 errors with suggested fixes; truncated, full text in `error_analysis.csv`).

ID	True→pred.	WER	Category	Suggested fix
ERR-01	hostel_mess→it_support	1.19	Phonetic & Lexical Substitution	Add Hinglish/Gujlish vocabulary expansion in tokenizer and train intent head on character-n-gra…
ERR-02	it_support→hostel_mess	1.00	ASR Decoder Hallucination (Script Shift)	Set condition_on_previous_text=False and compression_ratio_threshold=2.0 in Whisper decoding op…
ERR-03	student_services→hostel_mess	1.00	Acoustic Silence / Non-Speech Collapse	Prepend energy-based voice activity detection (VAD) filter to reject low-energy frames before W…
ERR-04	admissions→hostel_mess	1.00	ASR Decoder Hallucination (Script Shift)	Force Whisper language='en' decoding task with Latin script constraint to prevent unprompted Hi…
ERR-05	student_services→it_support	1.64	Lexical Cross-Talk & Hallucination	Use contextual embeddings (e.g. MiniLM / IndicBERT) that score entire query semantics rather th…
ERR-06	student_services→it_support	1.58	Semantic Hallucination Overwrite	Fine-tune Whisper acoustic decoder on target speaker acoustics to prevent severe phantom halluc…
ERR-07	admissions→hostel_mess	2.12	Semantic Hallucination Overwrite	Incorporate audio-level intent classification directly from Whisper encoder embeddings (end-to-…
ERR-08	hostel_mess→it_support	1.61	Semantic Keyword Hijack	Apply cross-attention acoustic adaptation on speaker cohort to ensure domain-specific hostel vo…
ERR-09	it_support→student_services	1.00	Cross-Domain Semantic Misalignment	Add domain-specific lexicon biasing during Whisper beam search decoding using campus vocabulary…
ERR-10	admissions→it_support	2.12	Lexical Confusion & Peripheral Tokens	Train joint ASR-Intent loss where acoustic encoder explicitly weights intent discriminative pho…
ERR-11	it_support→admissions	1.00	Cross-Intent Academic Overlap	Disambiguate academic deadline queries from administrative admissions via fine-grained multi-la…
ERR-12	hostel_mess→it_support	1.22	Acoustic Bleed & Academic Hallucination	Normalize Whisper prompt conditioning to bias toward residential and campus administration term…
ERR-13	it_support→student_services	1.00	Cross-Intent Academic Overlap	Weight domain-specific technical IT terms ('email forwarding', 'hpc', 'server') higher using we…
ERR-14	student_services→it_support	1.38	Acoustic Substitution & Technical Drift	Expand medical/counselling training samples in student_services intent class to increase semant…
ERR-15	admissions→hostel_mess	1.40	Severe Semantic Hallucination (Hostel Bias)	Fine-tune Whisper base decoder cross-attention layers on domain recordings to suppress random r…
ERR-16	hostel_mess→student_services	1.25	Composite Intent Ambiguity	Introduce composite intent schema or fallback routing to human agent when classification entrop…
ERR-17	it_support→admissions	0.97	Hallucinated Guest Accommodation Query	Fine-tune acoustic encoder on noisy lab acoustic environments with server room background rumbl…
ERR-18	student_services→hostel_mess	1.56	Academic Schedule Confusion	Calibrate probability thresholds per intent class to avoid majority class collapse on uncertain…
ERR-19	admissions→student_services	1.21	Multilingual Script Infiltration (Urdu/Hinglish)	Enforce strict Unicode Latin/Devanagari script filtering in ASR post-processing pipeline before…
ERR-20	admissions→student_services	1.06	Acoustic Mismatch & Keyword Ambiguity	Implement multi-task loss weighting and negative sample penalty on cross-domain vocabulary.…

Unpublished working draft. Not for distribution.